

Network Monitoring and Traffic Analysis Dashboard

Project Design, Implementation, Testing, and User Guide

Project name	NetScope
Application type	Local PC dashboard
Backend	Python, Flask, Scapy, SQLite
Frontend	HTML, CSS, JavaScript, Canvas charts
Report format	A4 portrait PDF

1. Project Overview

NetScope is a PC-oriented network monitoring dashboard that captures and analyzes network traffic in near real time. It provides a Wireshark-like packet list and details panel together with protocol charts, traffic volume, bandwidth statistics, host rankings, filters, alerts, and report export.

The application works in live-interface mode for authorized traffic capture and in sample mode for safe demonstration without capture permissions. Captured information remains on the computer by default.

2. System Architecture

The project separates capture, analysis, persistence, routes, reporting, presentation, and testing. A background worker captures traffic while the Flask application serves dashboard data. SQLite stores a bounded set of packet metadata. The browser requests updated data once per second and renders charts locally.

3. Functional Requirements Coverage

Requirement	Implementation
Live capture	Scapy capture from a selected interface
Packet parsing	IP addresses, protocol, ports, size, time, summary
Traffic metrics	Packets, bytes, packet rate, per-second timeline
Protocol breakdown	TCP, UDP, DNS, ICMP, and other
Host analysis	Sent/received totals, top talkers and listeners
Charts	Canvas traffic line chart and protocol doughnut chart
Filtering	IP, protocol, port, and time range
Alerts	Configurable byte-rate threshold
Exports	Filtered CSV and A4 PDF

4. Data and Privacy

The database stores packet metadata only. Payload content is not retained. Records remain in a local SQLite file and older rows are removed when the configured history limit is reached. The operator can clear all records from the dashboard.

5. Error Handling and Performance

Malformed packets are skipped and reported without terminating the complete application. Permission, interface, or driver errors appear in the capture status area. Capture runs in a daemon thread, while database indexes support common time, address, and protocol queries. API responses limit the displayed packet list so continuous capture does not freeze the interface.

6. Testing

Automated tests verify packet normalization, known metric totals, and successful dashboard loading with a temporary database. Manual tests cover sample capture, live capture, start and stop behavior, filter combinations, packet selection, host rankings, clearing stored records, threshold alerts, and CSV/PDF downloads.

7. Installation and Operation

Create a Python virtual environment, install requirements.txt, and run run.py. Open <http://127.0.0.1:5000> in a PC browser. Sample mode runs without special access. Live capture needs administrator/root privileges and may require Npcap on Windows.

8. Recommended PDF Format

A4 portrait is the recommended submission format. It prints reliably, fits standard university report requirements, and is comfortable to review on a laptop. The live PDF export uses 16 mm margins, compact summary tables, repeating column headings, and page breaks handled automatically for larger traffic reports.

9. Future Development

The modular design can be extended with PCAP input and output, saved filter profiles, IPv6-specific reporting, multiple alert rules, DNS query analysis, flow reconstruction, long-term aggregation, and user-configurable charts without redesigning the complete system.